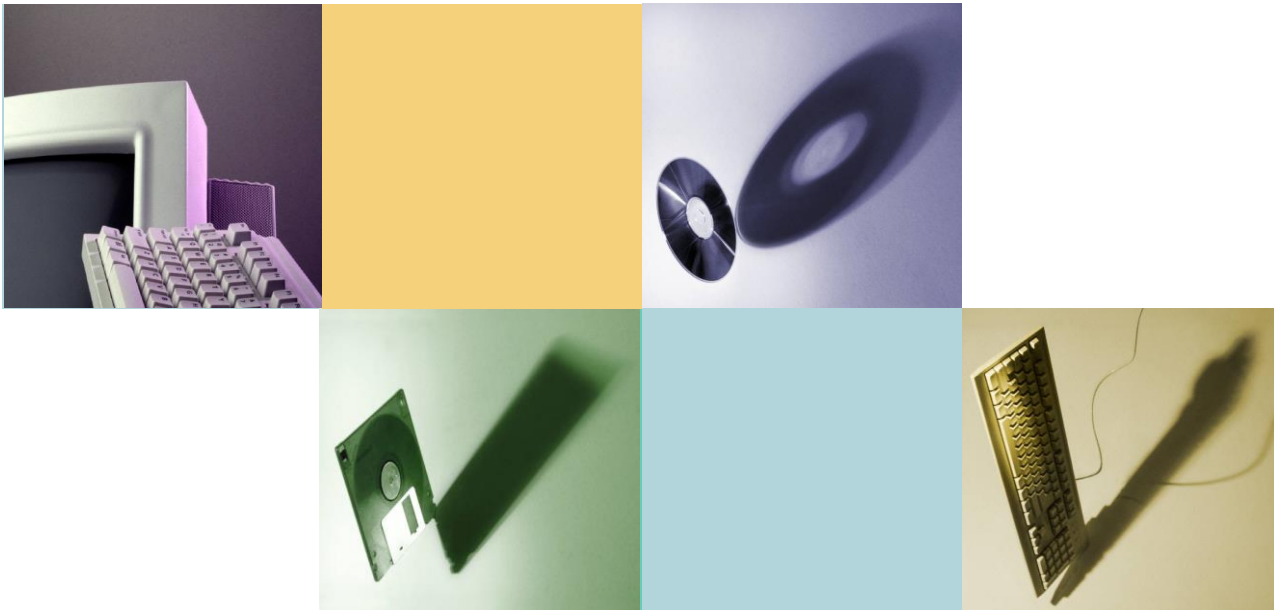


第4章 汇编语言程序设计



第4章 汇编语言程序设计

- 概述
- 汇编语言的程序格式和语句分类与格式
- 伪指令
- 宏指令
- 汇编语言程序设计
- DOS和BIOS系统功能调用
- 汇编语言与C++语言混合编程



4.1 概述

4.1.1 计算机语言的分类

◆ 机器语言：

优点：可以直接被计算机识别，执行速度快，占用内存空间少。

缺点：不直观，编写、阅读和修改都很繁琐。

◆ 汇编语言：

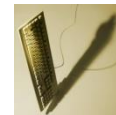
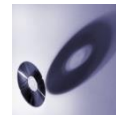
优点：执行速度较快，占用内存空间较少，编写、阅读和修改比较方便。

缺点：面向机器的语言，通用性差。

◆ 高级语言：

优点：编写、阅读和修改比较方便，通用性好。

缺点：执行速度慢，占用内存空间大，

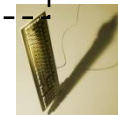
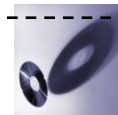


用C语言编程实现 $c = a + b$ ，并在屏幕上显示出结果。

例1

```
#include "stdio.h"
main( )
{
    int a,b,c;
    a=1;
    b=2;
    c=a+b;
    printf("c=%d\n",c);
}
```

编译后的目标文件达到3.59KB



例 2. $C = a + b$

```
data segment
a          db      ?
b          db      ?
c          db      ?
string     db      'c=$'
data ends
```

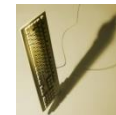
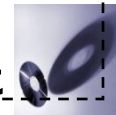
```
code segment
main proc far
    assume cs:code,
           ds:data,

start:
```

```
    push    ds
    sub     ax, ax
    push    ax
    mov     ax, data
    mov     ds, ax
```

汇编后的目标文件只有
208字节

```
    mov     a, 1
    mov     b, 2
    mov     al, a
    add     al, b
    mov     c, al
    lea     dx, string
    mov     ah, 09
    int     21h
    add     c, 30h
    mov     dl, c
    mov     ah, 2
    int     21h
    mov     dl, 0ah
    int     21h
    mov     dl, 0dh
    int     21h
    ret
main endp
code ends
end start
```



4.1.2 汇编语言程序的格式

DATA SEGMENT

.....

DATA ENDS

STACK SEGMENT

.....

STACK ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA,SS:STACK

START: MOV AX,DATA

MOV DS,AX

MOV AX,STACK

MOV SS,AX

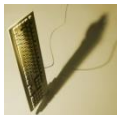
.....

MOV AH,4CH

INT 21H

CODE ENDS

END START



汇编语言程序具有如下特点：

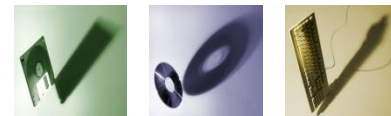
- 分段结构

汇编语言程序采用分段组织的方式，示例中包含两个段，**DATA**和**CODE**，以段定义伪指令**SEGMENT**表示段的开始，以段定义伪指令**ENDS**表示段的结束。

- 汇编语句

指示性语句：用于告诉汇编程序如何汇编，在可执行程序中，无可执行指令与其对应。

指令性语句：是执行语句，汇编语言程序对应的可执行程序，有可执行指令与其对应。



● 汇编语言和DOS的接口

方法一：将主程序定义成一个属性为**FAR**的过程，在程序的开始将**INT 20H**指令段地址(**DS**的值)和偏移地址(**0**)压入堆栈，

```
push    ds
sub     ax, ax
push    ax
```

在程序结束时用**RET**(将压入堆栈的值弹出到**IP**和**CS**)结束，相当于执行**INT 20H**指令，使程序正常结束，返回**DOS**

方法二：用**DOS**功能调用。

DOS功能调用中的**4CH**号调用，其功能是结束当前程序，返回**DOS**。其调用方法是在程序结束前加入以下两条指令。

```
MOV AH,4CH
INT 21H
```



4.2 汇编语言的程序格式和语句分类

4.2.1 汇编语言的程序格式

例：用汇编语言编程实现 $C=A+B$ 。

程序代码如下：

```
DATA SEGMENT
```

```
    A DB ?
```

```
    B DB ?
```

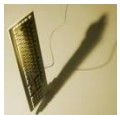
```
    C DB ?
```

```
DATA ENDS
```

```
STACK SEGMENT
```

```
    ST DB 200 DUP(?)
```

```
STACK ENDS
```



CODE SEGMENT

MAIN PROC FAR

ASSUME CS:CODE,DS:DATA

START:

PUSH DS

XOR AX,AX

PUSH AX

MOV AX,DATA

MOV DS,AX

MOV A,78

MOV B,-65

MOV AL,A

ADD AL,B

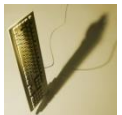
MOV C,AL

RET

MAIN ENDP

CODE ENDS

END START



对汇编语言程序结构的几点说明：

- 分段结构。至少包含代码段
- 变量的定义：通常在数据段和附加段完成。
- 堆栈段主要用于存放需要保护的数据，可以自己设置堆栈段，或由系统自动分配堆栈空间使用(建议)。
- **ASSUME**伪指令
指明程序逻辑段和存储器物理段之间的对应关系，但没有建立对应关系
- 用**END**伪指令结束整个源程序。



4.2.2 汇编语句的分类与格式

1.汇编语句的分类与格式

1) 指令性语句

指令性语句是可执行语句，指令性语句主要是指令系统中的指令。
指令性语句的格式是：

[名字] [指令前缀] 指令助记符 [操作数] [; 注释]

2) 指示性语句

伪指令是指示性语句，指示性语句不是可执行语句，
功能：主要是在汇编过程中告诉汇编程序如何汇编，

指示性语句的格式是：

[名字] 助记符 [操作数] [; 注释]



2.格式说明

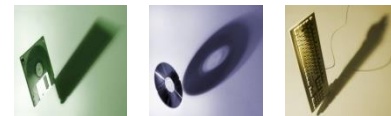
1) 名字

指令性语句中的名字是标号，与指令前缀或者助记符以“:”间隔。

指示性语句中的名字可以是变量名、段名、过程名等，与助记符之间以空格间隔。

2) 指令前缀

对于8086/8088系统中用到的前缀主要有段超越前缀，锁定前缀和重复前缀。



3) 助记符

指令性语句中的助记符是语句的关键字，用于指出指令的功能和操作，是指令中不能省略的部分。

指示性语句中的助记符是伪指令的关键字，用于规定汇编程序完成的操作，

4) 操作数

■ 常数

在指令的地址码字段直接给出，在指令中只能作为源操作数。
常见的常数形式有：数字型常数、字符型常数。



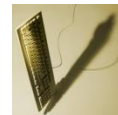
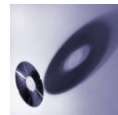
数字型常数：数字型常数可以是二进制数，十进制数，八进制数和十六进制数。

字符型常数：是由单引号引起的一个或一串字符，如 'C', 'ABC'等。在汇编时字符在存储器中以**ASCII**码存放。

■ 标号和变量

标号可以作为转移指令、过程调用指令等的操作数，标号表征的是关联指令的地址，具有段属性、偏移属性和类型属性。

变量可以作为大部分指令的操作数，变量具有段属性、偏移属性和类型属性。



例:

DS1 DB 35H, 6FH

MOV AL, DS1 ;变量DS1的值35H送至AL

MOV BX, OFFSET DS1

;将变量DS1的偏移地址送至BX,

;即数据35H的偏移地址送至BX

3) 由寻址方式给出的寄存器操作数或存储器操作数

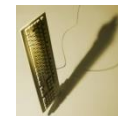


4) 表达式

表达式是由常数、标号、变量及各种寻址方式表示的操作数经**运算符**组合而成的，表达式的**求值是在汇编过程中进行的**。

运算符

- 算术运算符
- 关系运算符
- 逻辑运算符
- 分析运算符
- 属性运算符。



算术运算符有“+”，“-”，“*”，“/”，MOD(求余)，SHL(左移)和SHR(右移)七种。

算术运算执行的是整数运算，运算的结果为整数。

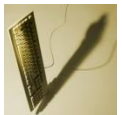
数值表达式中7种算术运算符都可以使用

地址表达式只能使用加法运算和减法运算，实现同一逻辑段的地址进行加法或减法运算。

例：

MOV AL,26MOD4 ;汇编后变为MOV AL, 2

MOV BH, (35*2+10)/7 ;汇编后变为MOV BH,11



关系运算符:

相等	不相等	小于	大于	小于等于	大于等于
EQ	NE	LT	GT	LE	GE

操作对象: 数值或者同一逻辑段的地址

关系运算的**结果**

- 关系**成立**时, 其结果为**全1**(8位为0FFH, 16位为0FFFFH)
- 关系**不成立**时, 其结果为**全0**(0)。

例:

MOV BX, 4EQ3 ; 汇编后变为MOV BX,0

MOV BX, 4NE3 ; 汇编后变为MOV BX,0FFFFH



逻辑运算符

逻辑非	逻辑与	逻辑或	逻辑异或
NOT	AND	OR	XOR

逻辑运算的操作对象为**数值**，不能对地址进行逻辑运算。

例：

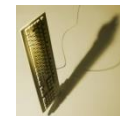
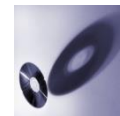
MOV AL,0D0H AND 55H ; 汇编后变为MOV AL,50H

MOV AL,0AEH OR 0AAH ; 汇编后变为MOV AL,0AEH



分析运算符主要用于分析操作对象的属性，操作对象可以是变量或者标号。

运算符	操作对象	功能
OFFSET	变量或标号	分析变量或者标号所在逻辑段的偏移地址
SEG	变量或标号	分析变量或者标号所在逻辑段的段地址
TYPE	变量或标号	分析变量或者标号的类型属性
LENGTH	变量	分析变量的长度
SIZE	变量	分析变量的大小



类型	BYTE (字节)	WORD (字)	DWORD (双字)	QWORD (8字节)	TBYTE (10字节)	NEAR	FAR
类型值	1	2	4	8	10	-1	-2

例:

DATA SEGMENT

VAR DW 1234H,5678H

ARRAY DD 12345678H

STR DB 12H,34H,56H,78H

DATA ENDS

CODE SEGMENT

...

MOV DX,OFFSET ARRAY ;将ARRAY的偏移地址0004H送至DX

MOV AX , TYPE VAR ; 2 → (AX)

MOV BX , TYPE ARRAY ; 4 → (BX)

MOV CX , TYPE STR ;1→ (CX)

...

CODE ENDS



例:

DATA SEGMENT

FEES DW 100 DUP (0) ;LENGTH FEES = 100

ARRAY DW 1,2,3 ; LENGTH ARRAY = 1

DATA ENDS

CODE SEGMENT

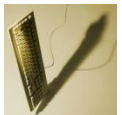
...

MOV AX , LENGTH FEES ;100 → (AX)

MOV BX , LENGTH ARRAY ;1 → (BX)

...

CODE ENDS



例：已知变量上例中定义的变量，利用**SIZE**运算符分析变量的大小。

MOV AX , SIZE FEES ;200→ (AX)

MOV BX , SIZE ARRAY ;2→ (BX)

5) 注释

汇编语言中“;”后的内容为注释部分，注释部分可有可无，在程序设计中，清晰有序的注释可大大增加程序可读性。



4.3.3 段定义伪指令

1.SEGMENT/ENDS

成对使用，用来指定段的名称和范围。

格式： **段名** **SEGMENT** [定位类型] [组合类型][**'类别'**]

本段程序内容

(指令或伪指令)

段名 **ENDS**

一致

段的标识符，命名规则
同变量、标号等

任选项告诉汇编程序和连接程序如何确定段的边界，以及如何组合几个不同的段等。



ASSUME 伪指令

功能：用来告诉汇编程序逻辑段和物理段之间的关系。

格式：

[]内的内容可有可无

ASSUME 段寄存器名：段名符[，段寄存器名：段名符，.....]

示例： **ASSUME CS: CODE, DS: DATA, SS: STACK**

注意：

- 1) ASSUME 语句必须安排在代码段内，一般应放在代码段之首。
- 2) ASSUME伪操作只是通知汇编程序有关段寄存器与逻辑段的关系，并没有给段寄存器赋予实际的初值。

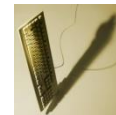
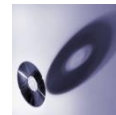


4.3.3 数据定义伪指令

数据定义语句为变量分配存储单元，变量名与第一个存储单元相联系。

语句格式

变量名	DB	表达式	——	定义字节数据
变量名	DW	表达式	——	定义字数据
变量名	DD	表达式	——	定义双字数据
变量名	DQ	表达式	——	定义长字数据
变量名	DT	表达式	——	定义十字节数据



操作数：操作数是变量定义时分配的存储空间中赋予的初值，可以是常数、字符串、表达式、变量和标号等。

1.操作数是一个或多个数值常量

如果是多个操作数时，操作数之间用,间隔。

例： **D1 DB 12H,25H**

;定义字节类型变量**D1**，分配两个存储单元，初值为**12H**， **25H**

D2 DW 1234H

;定义字类型变量**D2**，分配两个存储单元，初值为**34H**， **12H**

D3 DD 1A2B3C4DH

;定义双字类型变量**D3**，分配四个存储单元，初值为**4DH**， **3CH**， **2BH**,**1AH**



2.操作数是一个或者多个可求值的数值表达式

例: **DA DB 3*4, 5+6*2**

;定义字节类型变量**DA**, 分配两个存储单元, 初值为**12**和**17**

3.操作数是字符串常量

例: **D4 DB '12'**

;定义字节类型变量**D4**, 分配两个存储单元, 初值为**31H**, **32H**

D5 DW '12'

;定义字类型变量**D5**, 分配两个存储单元, 初值为**32H**, **31H**



4.操作数部分为?

?可以用于**DB、DW、DD**类型的变量定义中，其作用是**给定义的变量分配存储空间，但是不赋予初值**，一般用于定义存放结果的变量。

例：**DA1 DB ?**

；定义字节类型变量**DA1**，分配**1**个存储单元，不赋初值

DA2 DD ?

；定义字节类型变量**DA2**，分配**4**个存储单元，不赋初值



5.操作数部分为带**DUP**的表达式

DUP表达式格式: 重复次数 **DUP** (表达式)

DUP表达式功能: 表达式重复预置, 预置次数由重复次数确定

例:

TTA DB 50 DUP (0)

; 分配**50**个存储单元, 预置初值为**0**

TTB DW 100 DUP (?)

; 分配**200**个存储单元, 未预置初值

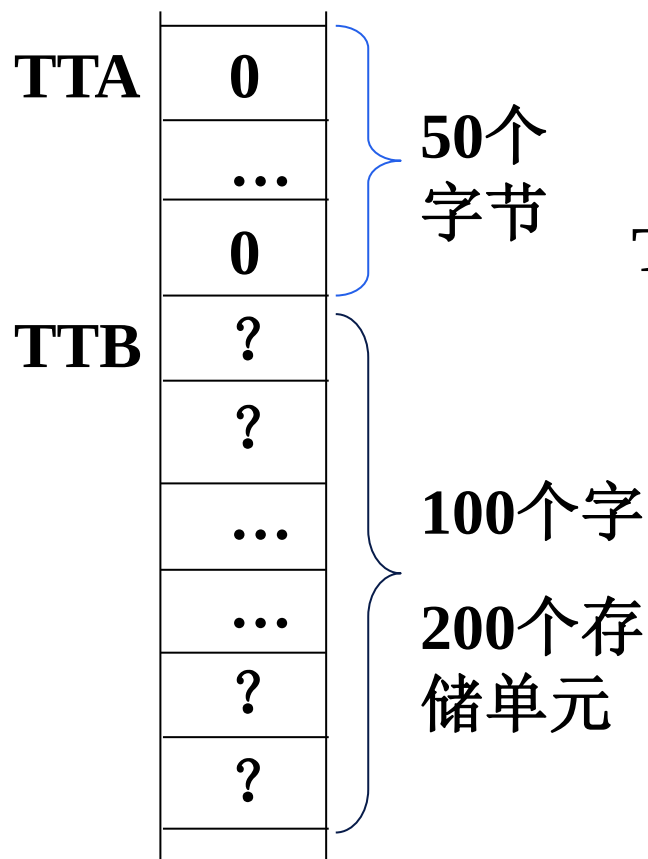
TTC DB 10 DUP ('ABC', 0BH)

; 分配**40**个存储单元, 重复预置初值**41H**

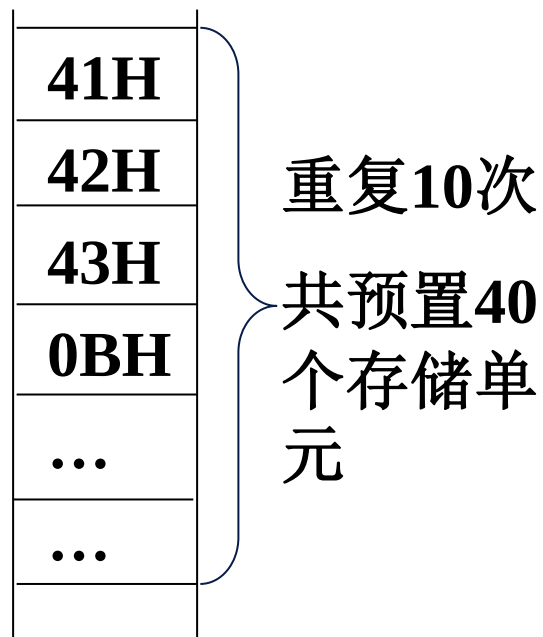
; **42H, 43H, 0BH**, 重复预置**10**次



内存分配:



TTC



注: TTC接TTB
地址连续



6.操作数是地址表达式

1)用**DW**定义时，把操作数的偏移地址赋给变量。

A1 DW VALUE；定义**A1**为**VALUE**的偏移地址

2) 用**DD**定义时，低位字用于预置偏移地址，高位字用于预置段地址

A2 DD VALUE

；**A2**的高位字为**VALUE**的段地址，低位字为**VALUE**的偏移地址

3) 操作数中的变量与标号可与常数相加减，相当于偏移地址加减常数，段地址不变

4) 变量与变量，标号与标号不能相加，可相减，结果为纯数值。



例:

ZERO DB 0

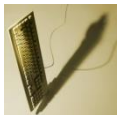
ONE DW 1234H,5678H

TWO DW ZERO

THREE DD TWO

FOUR DB TWO-ONE

ZERO		0500H:0100H
	0	
ONE	34H	
	12H	
TWO	78H	
	56H	
	00H	
	01H	
THREE	05H	
	01H	
	00H	
FOUR	05H	
	04H	



关于变量的使用:

1) 变量的类型必须与指令的要求相符

例: `MOV AL, ONE` 错
`MOV AX, ONE` 对

2) 变量对应于第一个数据项, 其它数据项应用地址表达式或其它寻址方式。

例: `MOV AX, ONE+2`

`MOV BX, OFFSET ONE`
`MOV AX, [BX+2]`



4.3.5 符号定义伪指令

等值语句EQU

格式： 符号名 EQU 表达式

功能：给EQU后的表达式赋予EQU前的符号名

例：

XX EQU 2010H ;给常数2010H赋予一个符号名XX

ADS EQU [BX+80H] ;给地址表达式[BX+80H]赋予一个符号名ADS

LOD EQU MOV ;给指令MOV赋予一个符号名LOD

注意：利用EQU定义的符号，在未经解除前不能重复定义。



2. 等号语句=

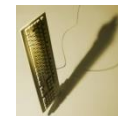
格式： 符号名 = 表达式

功能： 等号语句的功能与**EQU**语句的功能完全相同，=语句与**EQU**语句的差别在于使用=语句定义的符号可以重复定义。

例：

AA=15 ;定义AA，AA的值为15

AA=25+AA ;重新定义AA，AA的值为40



3.LABEL

格式： 名字 LABEL 类型

功能： 定义变量或标号的类型。

例：

```
DBUFFER1 LABEL WORD
```

```
DBUFFER2 DB 20 DUP (0)
```



4.3.6 过程定义伪指令

格式:

过程名

PROC [NEAR] (FAR)

类型说明符，说明过程的属性
(段内或段间)

必须有且一致

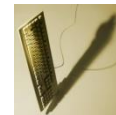
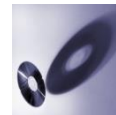
过程名

⋮
⋮
RET

ENDP

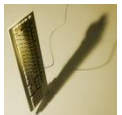
子程序

如为NEAR属性，
可省略



例：过程XX的定义示例，过程XX与主调程序在同一个逻辑段MYCODE。

```
MYCODE    SEGMENT
XX        PROC NEAR
          DEC CX
          RET
XX        ENDP
START:    MOV CX, 0FFAH
          |
          CALL XX
          |
MYCODE    ENDS
```



例：过程XX的定义示例，过程XX与主调程序在不在同一个逻辑段。

MYCODE1 SEGMENT

...

XX PROC FAR

...

RET

XX ENDP

...

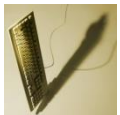
- **MYCODE1 ENDS**

MYCODE2 SEGMENT

CALL XX

...

- **MYCODE2 ENDS**



4.3.8 程序计数器与定位伪指令

1. 程序计数器\$

“\$”表示当前位置的偏移地址。

例：

```
DATA SEGMENT
```

```
    A DB 10H,20H,30H,40H,50H
```

```
    ;A的偏移地址为0000H，占用了5个存储单元
```

```
    LENGTHA EQU $-A
```

```
    ; LENGTHA的值为0005H-0000H=0005H
```

```
DATA ENDS
```



2.ORG

格式: **ORG** 表达式

功能: 将表达式的值作为后续数据或者指令的偏移地址

例:

DATA SEGMENT

ORG 200H

ST1 DB 10H, 20H, 30H

LENGTH EQU \$-ST1

ST2 DW ?

ORG 400H

ST3 DW 123H, 456H

;给变量**ST3**在偏移地址为**400H**的位置分配存储单元

DATA ENDS



3. EVEN

格式: **EVEN**

功能: 规定后续程序或数据从偶地址开始存放。当默认地址为偶数时, 不做调整, 默认地址为奇数时, 则偏移地址加1, 指向后续的偶地址单元。

例:

```
DATA SEGMENT
```

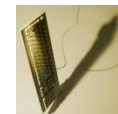
```
    D1 DB 'A'
```

```
    EVEN
```

;当前偏移地址为0001H, 调整, 偏移地址为0002H

```
    D2 DW 1234H
```

```
DATA ENDS
```



4.3.9 结束伪指令

标志着整个源程序的结束。

格式为: **END** <表达式>

DATASEG	SEGMENT
XX1	DW 1234H
YY1	DW 4567H
SULT	DW ?
DATASEG	ENDS
CODESEG	SEGMENT
	ASSUME CS: CODESEG, DS: DATASEG
START:	MOV AX, DATASEG
	MOV DS, AX
	MOV AX, XX1
	SUB AX, YY1
	MOV SULT, AX
CODESEG	ENDS
	END START



4.3 汇编语言程序设计

4.5.1 汇编语言程序设计步骤

- 分析问题。根据实际问题抽象出其数学模型。
- 确定算法。
- 根据算法绘制流程图
- 分配存储空间和工作单元
- 编写程序并进行静态检查
- 上机调试



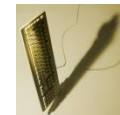
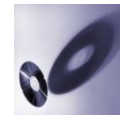
4.5.2 顺序结构程序设计

例：在内存数据区**2100H**单元存有**2**位压缩**BCD**码，将其变成非压缩**BCD**码，低位存于**2100H**单元，高位存于**2101H**单元。

分析：**2100H**单元存放的**2**位**BCD**码，需要将**2**位数分离

分离出低**4**位的方法：就是数据与**0FH**进行逻辑与运算

分离出高**4**位的方法：是将数据右移**4**位。



程序代码如下：

DATA SEGMENT

ORG 2100H

ZBCD DB 56H

FBCD DB ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA

START:

MOV AX,DATA

MOV DS,AX

LEA BX,ZBCD

MOV AL,[BX]

AND ZBCD ,0FH

MOV CL,4

SHR AL,CL

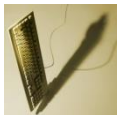
MOV [BX+1],AL

MOV AH,4CH

INT 21H

CODE ENDS

END START



例：已知某班微机原理课程的成绩按照学号从小到大的顺序排列在**SCOCE**表格中，要查的学生学号存放在变量**NO**中，查找学生的成绩，将成绩存放到**RESULT**单元。

程序代码如下：

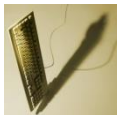
DATA SEGMENT

SCORE DB 85,76,67,57,82,74,92,95,83,68

NO DB 7

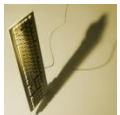
RESULT DB ?

DATA ENDS



```
CODE SEGMENT  
ASSUME CS:CODE,DS:DATA  
START :  
MOV AX,DATA  
MOV DS,AX  
LEA BX,SCORE  
MOV AL,NO  
DEC AL  
XLAT  
MOV RESULT,AL  
MOV AH,4CH  
INT 21H  
CODE ENDS  
END START
```

```
LEA SI,SCORE  
MOV BL,NO  
MOV BH,0  
DEC BX  
MOV AL,[BX+SI]  
MOV RESULT,AL
```



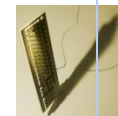
4.5.3 分支结构程序设计

例：将键盘输入的一个小写字母用大写字母形式在屏幕上显示出来。

程序代码如下：

```
CODE SEGMENT  
ASSUME CS:CODE  
MAIN PROC FAR  
START:  
PUSH DS  
MOV AX,0  
PUSH AX  
MOV AH,1  
INT 21H
```

```
CMP AL, 'a'  
JB OVERFLOW  
CMP AL, 'z'  
JA OVERFLOW  
SUB AL,20H  
MOV DL,AL  
MOV AH,06H  
INT 21H  
OVERFLOW: RET  
MAIN ENDP  
CODE ENDS  
END START
```



例：内存数据区自**1000H**开始存放了**3**个带符号字节数据，将其中的最大值送至其后的**RESULT**单元。

程序代码如下：

DATA SEGMENT

ORG 1000H

BUF DB 13H,89H,76H

RESULT DB ?

DATA ENDS

CODE SEGMENT

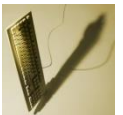
ASSUME CS:CODE,DS:DATA

START: MOV AX,DATA

MOV DS,AX



```
MOV BX,1000H
MOV AL,[BX]
INC BX
CMP AL,[BX] ;第1, 2个数据进行比较
JGE NEXT
MOV AL,[BX]
NEXT: INC BX
      CMP AL,[BX] ;将AL的值(前2个数中的大数与第3个数比较)
      JGE EXIT
      MOV AL,[BX]
EXIT:  MOV RESULT,AL
MOV AH,4CH
INT 21H
CODE ENDS
END START
```



例4-34： 编写程序实现符号函数的功能。

$$\text{符号函数: } Y = \begin{cases} 1 & X > 0 \\ 0 & X = 0 \\ -1 & X < 0 \end{cases}$$

程序代码如下：

DATA SEGMENT

X DB -8

Y DB ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA

START: MOV AX,DATA

MOV DS,AX



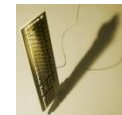
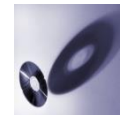
```
MOV AL,X
AND AL,AL
JS NEGA
JZ ZERO
MOV Y,1
JMP EXIT
NEGA:MOV Y,-1
JMP EXIT
ZERO: MOV Y,0
EXIT:MOV AH,4CH
INT 21H
CODE ENDS
END START
```

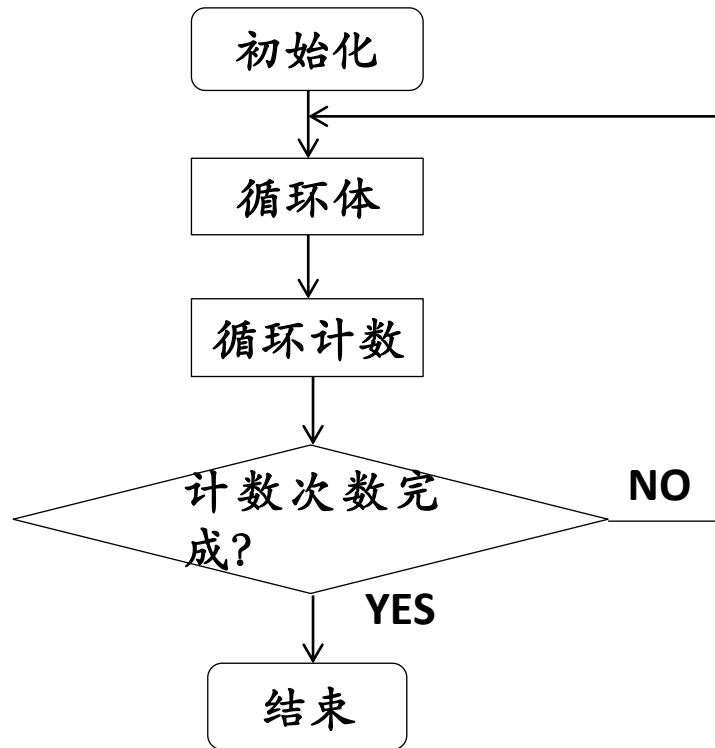


4.5.4 循环结构程序设计

循环结构程序通常包含**3**部分：循环初始条件，循环体，循环结束条件。

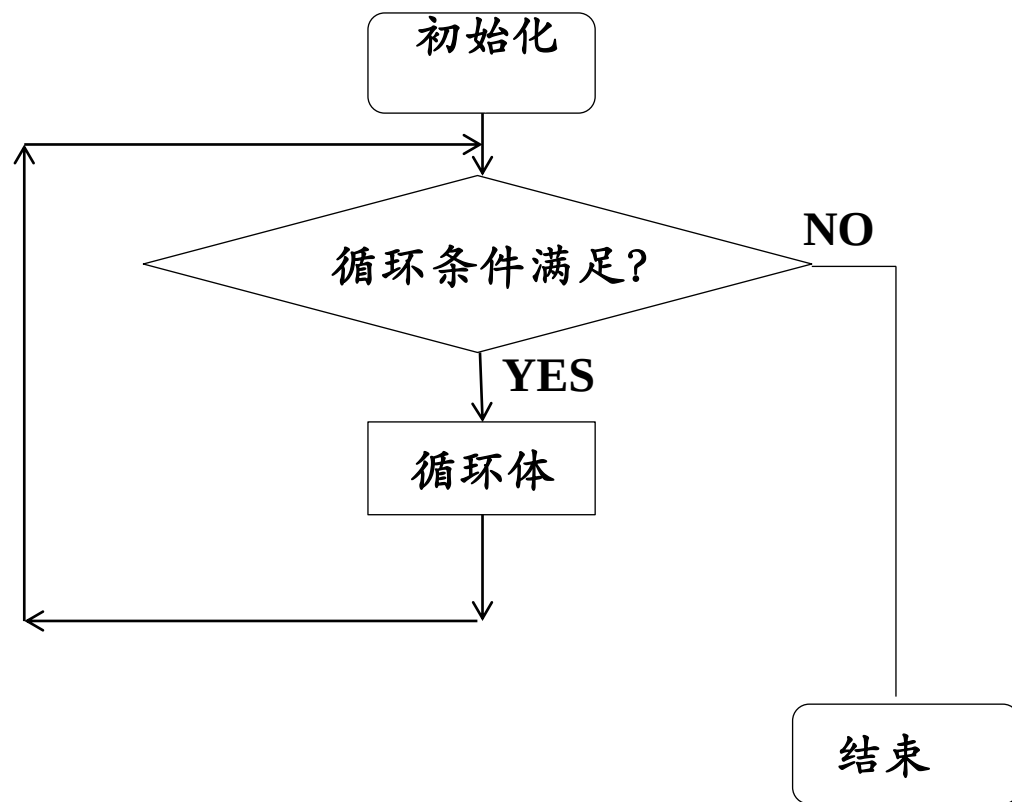
- **循环初始条件**：是指循环体执行前的初始状态，通常通过初始化寄存器和存储单元来完成。
- **循环体**：循环体是循环程序的核心部分，包括循环工作部分和循环控制部分。
- **循环结束条件**：在循环程序设计中必须给出循环结束条件，控制循环的结束，不能结束的循环为死循环。





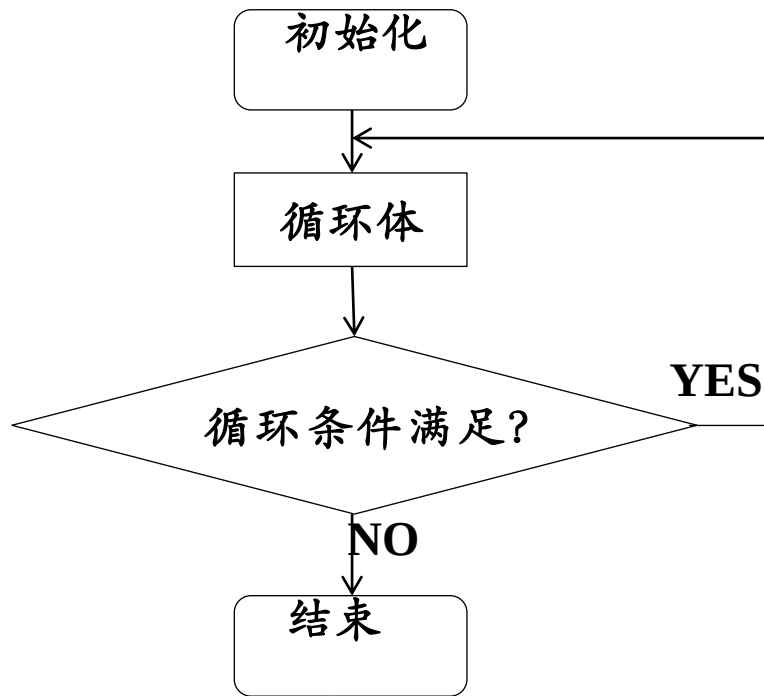
LOOP型循环执行过程



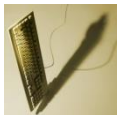


WHILE型循环执行过程





UNTIL型循环执行过程



例：内存数据去自1000H单元开始的10个存储单元存储的10个带符号字节数据，编程统计其中正数、负数和零的个数，并存放在其后的POSINUM，NEGNUM和ZERONUM单元中。
程序代码如下：

DATA SEGMENT

ORG 1000H

BUF DB 10H,23H,97H,0F3H,78H,94H,48H,0A0H,98H,54H

COUNT EQU \$-BUF

POSINUM DB ?

NEGNUM DB ?

ZERONUM DB ?

DATA ENDS

CODE SEGMENT

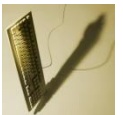
ASSUME DS:DATA,CS:CODE

MAIN PROC FAR

START: PUSH DS

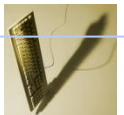
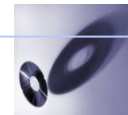
MOV AX, 0

PUSH AX



```
MOV AX,DATA
MOV DS,AX
MOV BX,OFFSET BUF
MOV CX,COUNT
MOV AH,0
MOV DX,0
AGAIN: MOV AL,[BX]
      AND AL,AL
      JZ  ZERO
      JS  NEGATIVE
      INC AH
      JMP NEXT
NEGATIVE: INC DL
      JMP NEXT
ZERO: INC DH
NEXT: INC BX
      LOOP AGAIN
```

```
MOV POSINUM ,AH
MOV NEGNUM,DL
MOV ZERONUM ,DH
      RET
MAIN ENDP
CODE ENDS
END START
```



例：在一串给定个数的带符号字数据中寻找最大值，将最大值和最大值地址分别存放至MAX和MAXADDR存储单元。

DATA SEGMENT

BUF DW

**12H,253AH,9036H,548AH,8778H,503BH,9388H,318CH,
0FA43H,655BH**

LEN EQU \$-BUF

MAX DW ?

MAXADDR DW ?

DATA ENDS

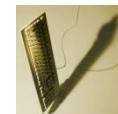
CODE SEGMENT

ASSUME CS:CODE,DS:DATA

MAIN PROC FAR

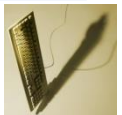
START:MOV AX,DATA

MOV DS,AX



```
LEA BX,BUF
MOV CX,LEN
SHR CX,1
MOV AX,[BX]
MOV SI,BX
INC BX
INC BX
DEC CX
AGAIN: CMP AX,[BX]
JGE NEXT
MOV AX,[BX]
MOV SI,BX
NEXT: INC BX
      INC BX
      LOOP AGAIN
```

```
MOV MAX,AX
MOV MAXADDR, SI
MOV AH,4CH
INT 21H
MAIN ENDP
CODE ENDS
END START
```



例：假设内存数据区自3000H开始存有一串以'\$'结束的字符串，编程统计其中的'#'的个数并显示。

程序代码如下：

DATA SEGMENT

ORG 3000H

STR DB 'INV0&FAL2V#J76LH###TT\$'

DATA ENDS

CODE SEGMENT

ASSUME DS:DATA,CS:CODE

MOV AX,DATA

MOV DS,AX

LEA SI,STR

MOV BX,0



CHECK: CMP BYTE PTR[SI], '\$'

JZ OUTPUT

NEXT: CMP BYTE PTR[SI], '#'

JNE NE

INC BX

NE: INC SI

JMP CHECK

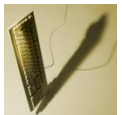
OUTPUT: CMP BX,10

JB OP_DIGIT

MOV AX,BX

MOV CL,10

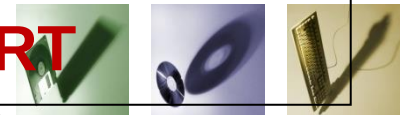
DIV CL



```
MOV BX,AX
ADD AL,30H
MOV AH,6
INT 21H
ADD BH,30H
MOV DL,BH
MOV AH,6
INT 21H
JMP EXIT
OP_DIGIT: ADD BL,30H
MOV DL,BL
MOV AH,6
INT 21H
EXIT: MOV AH,4CH
INT 21H
CODE ENDS
END START
```

代码可精简为

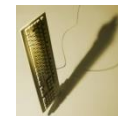
```
MOV BX,AX
ADD AL,30H
MOV AH,6
INT 21H
MOV BL,BH
OP_DIGIT: ADD BL,30H
MOV DL,BL
MOV AH,6
INT 21H
EXIT: MOV AH,4CH
INT 21H
CODE ENDS
END START
```



4.6 DOS和BIOS系统功能调用

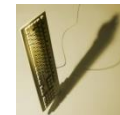
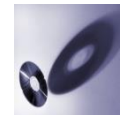
4.6.1 DOS系统功能调用

- 设备管理类
- 目录管理类
- 文件管理类
- 其他类



DOS系统功能调用一般遵循如下3个步骤：

- 1 按照DOS系统功能调用的入口参数要求送入口参数，无入口参数的忽略该步骤。**
- 2 将DOS系统功能调用的子程序编号送至AH。**
- 3 给出DOS子程序请求中断指令，即INT 21H。**



1.从键盘输入一个字符并回显(1号调用)

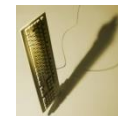
功能：等待从键盘输入一个字符，直到有键按下，有键按下后，在显示器上显示该字符，同时将该键的**ASCII**码送至**AL**。

入口参数：无

出口参数：键盘上按下键的**ASCII**码送至**AL**，并在显示器上显示该字符。

例：

```
MOV AH,1    ;功能号送AH  
INT 21H     ;DOS功能调用
```

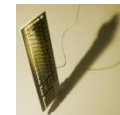
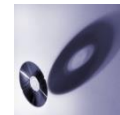


3.从键盘输入一个字符串(10号调用)

功能：将从键盘输入的以回车结束的一串字符送至指定的存储区域。

入口参数： **DS: DX**指向接收字符串的存储区的首存储单元，接收字符串的存储区的第一个字节存入用户设置的接收存储区可接收的最大字符数(含回车)。

出口参数： 将实际输入的字符串的**字符个数**(不含回车)存放到接收字符串存储区的**第二个字节存储单元**，实际输入的**字符串**从接收字符串存储区的**第三个存储单元开始存放**。



例:

DATA SEGMENT

BUF DB 20 ;用户设置的接收字符数
DB ? ;预留单元，接收实际输入的字符数
DB 20DUP(?) ;预留单元，接收输入的字符串

DATA ENDS

CODE SEGMENT

...

MOV AX,DATA

MOV DS,AX ;DS指向接收字符串存储区的首存储单元

MOV DX,OFFSET BUF

MOV AH,10 ;子程序编号送至AH

INT 21H ;DOS功能调用

...

CODE ENDS

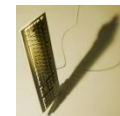
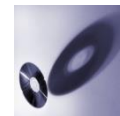


5.在显示器上显示一个字符串(9号调用)

功能：在显示器上显示一个字符串(字符串必须以'\$'结束，'\$'不显示)。

入口参数： DS: DX指向以'\$'结尾的字符串的首字符对应的存储单元。

出口参数： 无。



例:

DATA SEGMENT

STR DB 'Hello World!\$' ;以'\$'结束的字符串

DATA ENDS

CODE SEGMENT

...

MOV AX,DATA

MOV DS,AX

;DS指向字符串首字符对应的存储单元

MOV DX,OFFSET STR ;DX指向字符串首字符对应的存储单元

MOV AH,9

;子程序编号送至AH

INT 21H

;DOS功能调用

...

CODE ENDS



6. 键盘输入字符/显示器输出字符(6号调用)

功能：从键盘输入一个字符或者在显示器上输出一个字符。

输入功能时：

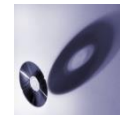
入口参数：将**0FFH**送至**DL**，表示从键盘输入一个字符。

出口参数：如果有键按下，**ZF=0**，按下字符的**ASCII**码送至**AL**寄存器；如果没有键按下，**ZF=1**。

输出功能时：

入口参数：将要输出字符的**ASCII**码送至**DL**(不能为**0FFH**)。

出口参数：无。



例：6号调用从键盘输入一个字符

MOV DL,0FFH	;入口参数0FFH送至DL，表示输入功能
MOV AH,6	;子程序编号送AH
INT 21H	;DOS功能调用

例：6号调用在显示器上输出一个字符

MOV DL, 'A'	;入口参数设置，'A'的ASCII码送至DL
MOV AH,6	;子程序编号送至AH
INT 21H	;DOS功能调用



例：从键盘输入一串小写字母，将其改为大写字母后输出。

程序代码如下：

DATAS SEGMENT

STRING1 DB 'Please input some small letters \$'

STRING DB 'THE CONVERTED LETTER: \$'

BUFF DB 100

DB ?

DB 100 DUP (?)

DATAS ENDS

CODES SEGMENT

ASSUME CS:CODES,DS:DATAS

START:

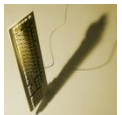
MOV AX,DATAS

MOV DS,AX

MOV DX,OFFSET STRING1

MOV AH,09H

INT 21H



MOV DX,OFFSET BUFF

MOV AH,0AH

INT 21H

MOV DX,OFFSET STRING

MOV AH,09H

INT 21H

MOV AH,06H

XOR SI,SI

MOV CL,BUFF[1]

L1:MOV DL,BUFF[SI+2]

CMP DL,'a'

JB L2

CMP DL,'z'

JA L2

SUB DL,20H

L2:INT 21H

INC SI

DEC CL

JNZ L1

MOV AH,4CH

INT 21H

CODES ENDS

END START

